PLATEFORME SAAS COCOTAILLE & COCOMOUSSO

GUIDE TECHNIQUE FRONTEND :

TEMPS RÉEL, NOTIFICATIONS, PUSH FCM & SMS OTP

Version API : v1 (NestJS 12 / Socket.IO / BullMQ / Firebase Admin / Termii)

Destiné aux équipes de développement Mobile (Flutter, React Native) et Web (Next.js, Écrans TV)

1. Vue d'Ensemble & Architecture de Communication

La plateforme Coco dispose d'une infrastructure asynchrone hautement optimisée conçue pour éliminer la latence dans les salons, automatiser l'appel des tickets, informer les clients sans délai et sécuriser les accès téléphoniques en Côte d'Ivoire sans générer de coûts superflus.

Canal / Protocole	Technologie Sous-Jacente	Cible Frontend	Cas d'Usage Clés
Temps Réel (WebSocket)	Socket.IO Namespace: /queue	Écrans TV Salons, App Mobile Client, Dashboard Gérant	Appel des tickets au micro/carillon, mouvement file d'attente, début et fin de prestation en direct.
Notifications In-App & Centre d'Alertes	PostgreSQL + Prisma + Relais WebSocket	App Mobile Client, Espace Coiffeur	Historique des alertes, pastille rouge non-lue, notifications d'acompte, de retard et de solde.
Notifications Push (FCM)	Firebase Admin SDK Multicast HTTP v1	Smartphones iOS et Android	Alertes réveillant l'appareil fermé : ticket prêt, rappel de RDV 2h avant, annulation, offre promo.
Vérification SMS OTP (100% Gratuit)	Firebase Phone Auth (SDK Client Google)	App Mobile Client et Web Auth	Validation du numéro de téléphone et réinitialisation de mot de passe (10 000 SMS/mois offerts par Google).
SMS Transactionnel (Opérateurs CI)	Termii SMS Gateway (+225 Orange, MTN, Moov)	Backend Automatique (Workers BullMQ)	SMS critiques de rappel 2h avant RDV, avertissement No-Show et confirmation de réservation payée.

PRINCIPE CLÉ POUR LE FRONTEND

Le frontend ne doit jamais déclencher manuellement l'envoi d'un SMS payant. Les SMS transactionnels sont gérés à 100% par le backend via BullMQ. En revanche, pour la réinitialisation de mot de passe, le frontend utilise directement Firebase Phone Auth pour bénéficier du quota 100% gratuit de Google.

2. Module 1 : Temps Réel (WebSockets & Socket.IO)

La passerelle QueueGateway écoute sur le namespace dédié /queue. Elle supporte deux modes d'utilisation distincts : les clients mobiles connectés avec leur compte utilisateur, et les écrans TV installés dans les salons de coiffure fonctionnant en mode public.

Paramètres de Connexion

• **URL du Gateway :** wss://api.votredomaine.com/queue (ou ws://localhost:3000/queue en local)

• **Namespace Socket.IO :** /queue (très important : ne pas se connecter sur le namespace racine /)

• **Transports recommandés :** ['websocket', 'polling'] (privilégier le transport websocket pour une latence < 10ms)

Les Deux Modes de Connexion

1. Mode Utilisateur Connecté (Client Mobile, Coiffeur, Gérant) : Transmettez le jeton JWT dans handshake.auth.token. Le serveur vérifie le jeton à la connexion et fait automatiquement rejoindre le socket à la salle customer:<userId>. Les alertes nominatives sont reçues instantanément.

2. Mode Écran TV Public de Salon : Aucun jeton requis ! Passez simplement le paramètre ?salonId=<uuid> dans la query de handshake. Le serveur fait automatiquement rejoindre l'écran à la salle salon:<salonId>.

Tableau des Événements Émis par le Serveur

Nom de l'Événement	Destinataires	Contenu du Payload	Action Frontend Recommandée
ticket:called	salon:<salonId> (Écran TV Salon)	{ type: "TICKET_CALLED", ticketNumber: "W-015", graceMinutes: 10, playChime: true }	Faire clignoter le ticket sur la TV, afficher la fenêtre modale et jouer le son carillon (chime.mp3).
ticket:called	customer:<userId> (Smartphone Client)	{ type: "YOUR_TURN", ticketNumber: "W-015", message: "C'est votre tour !" }	Vibration du smartphone, popup modale avec compte à rebours de grâce (10 min) et bouton de confirmation.
ticket:started	salon:<salonId>	{ type: "TICKET_STARTED", ticketNumber: "W-015", startedAt: "..." }	Déplacer le ticket de la liste d'attente vers la section "Prestations En cours" sur la TV et le dashboard.
ticket:completed	salon:<salonId>	{ type: "TICKET_COMPLETED", ticketNumber: "W-015" }	Retirer le ticket de la file active.
ticket:no_show	salon:<salonId>	{ type: "TICKET_NO_SHOW", ticketNumber: "W-015" }	Marquer le ticket en rouge (Non-présentation) avec mention de pénalité.
queue:updated	salon:<salonId>	{ action: "CREATED|CALLED|...", ticketId: "uuid" }	Invalider le cache React Query ou réexécuter l'appel GET /api/v1/salons/:id/queue.
notification:new	customer:<userId>	NotificationEntity (id, title, body, data, createdAt)	Incrémenter le badge rouge de la cloche (+1) et afficher un toast personnalisé au premier plan.

Messages Émis par le Client vers le Serveur

• **join:salon :** Payload: { salonId: string } — Permet à un client de changer de salon écouté dynamiquement.

• **leave:salon :** Payload: { salonId: string } — Quitte l'écoute des tickets d'un salon.

• **join:customer :** Payload: { customerId: string } — Permet une écoute explicite si le handshake était non authentifié.

Exemple d'Intégration React / React Native (Custom Hook)

Ce hook gère la connexion persistante, la reconnexion automatique, l'abonnement aux salles et le son audio du carillon :

```
import { useEffect, useRef, useState } from 'react';
import { io, Socket } from 'socket.io-client';

export function useQueueSocket(salonId?: string, userToken?: string) {
  const socketRef = useRef<Socket | null>(null);
  const [lastCalled, setLastCalled] = useState<any>(null);
  const [refreshTrigger, setRefreshTrigger] = useState(0);

  useEffect(() => {
    const socket = io('https://api.votredomaine.com/queue', {
      auth: userToken ? { token: `Bearer ${userToken}` } : undefined,
      query: salonId ? { salonId } : undefined,
      transports: ['websocket'],
      reconnection: true,
      reconnectionAttempts: 10,
      reconnectionDelay: 1000,
    });

    socketRef.current = socket;

    socket.on('connect', () => {
      console.log('Connecté au WebSocket Coco Queue:', socket.id);
      if (salonId) socket.emit('join:salon', { salonId });
    });

    socket.on('ticket:called', (payload) => {
      setLastCalled(payload);
      if (payload.playChime) {
        new Audio('/sounds/chime.mp3').play().catch(() => {});
      }
    });

    socket.on('queue:updated', () => setRefreshTrigger((prev) => prev + 1));

    return () => {
      socket.disconnect();
    };
  }, [salonId, userToken]);

  return { socket: socketRef.current, lastCalled, refreshTrigger };
}
```

Exemple d'Intégration Flutter (Dart)

Implémentation avec le package officiel socket_io_client et audioplayers :

```
import 'package:socket_io_client/socket_io_client.dart' as IO;
import 'package:audioplayers/audioplayers.dart';

class QueueSocketService {
  late IO.Socket _socket;
  final AudioPlayer _audioPlayer = AudioPlayer();

  void connect({String? salonId, String? userJwt}) {
    _socket = IO.io('https://api.votredomaine.com/queue', IO.OptionBuilder()
      .setTransports(['websocket'])
      .enableAutoConnect()
      .enableReconnection()
      .setAuth(userJwt != null ? {'token': 'Bearer ' + userJwt} : {})
      .setQuery(salonId != null ? {'salonId': salonId} : {})
      .build());

    _socket.onConnect((_) => print('Connecté au WebSocket Coco Queue'));

    _socket.on('ticket:called', (data) async {
      print('Ticket appelé : ${data['ticketNumber']}');
      if (data['playChime'] == true) {
        await _audioPlayer.play(AssetSource('sounds/chime.mp3'));
      }
    });

    _socket.on('notification:new', (notif) {
      print('Notification In-App reçue : ${notif['title']}');
    });
  }

  void disconnect() => _socket.disconnect();
}
```

3. Module 2 : Notifications In-App & Préférences

Le système de notifications In-App repose sur une double approche : la réception instantanée via WebSocket (événement notification:new) dès qu'une notification survient, et la synchronisation persistante via les endpoints REST (historique, compteur de non-lus, marquage comme lu).

Répertoire Complet des Endpoints REST Notifications

Méthode & Route	Authentification	Description & Rôle	Exemple Réponse
GET /api/v1/notifications	Bearer JWT (Requis)	Historique paginé des notifications du compte connecté. Supporte query params: page, limit, isRead, salonId.	{ total: 12, notifications: [{ id: "uuid", title: "...", isRead: false }] }
GET /api/v1/notifications/unread-count	Bearer JWT (Requis)	Retourne le nombre exact de notifications non lues. Idéal pour afficher la pastille rouge sur la cloche à l'ouverture de l'app.	{ unreadCount: 3 }
PATCH /api/v1/notifications/:id/read	Bearer JWT (Requis)	Marque une notification spécifique comme lue dès que l'utilisateur clique dessus.	{ id: "uuid", isRead: true, readAt: "2026-10-20..." }
PATCH /api/v1/notifications/read-all	Bearer JWT (Requis)	Marque l'ensemble des notifications non lues de l'utilisateur comme lues en une seule requête.	{ count: 3 }
GET /api/v1/notifications/preferences	Bearer JWT (Requis)	Récupère les préférences de réception des notifications de l'utilisateur pour chaque canal.	{ pushEnabled: true, smsEnabled: true, inAppEnabled: true, whatsappEnabled: true }
PATCH /api/v1/notifications/preferences	Bearer JWT (Requis)	Met à jour les préférences de canaux de l'utilisateur. Corps JSON avec les booléens modifiés.	{ smsEnabled: false }

STRATÉGIE DE MISE À JOUR DU BADGE CLOCHE

Au lancement de l'application : appelez GET /api/v1/notifications/unread-count. Lorsque l'application est ouverte et que le WebSocket reçoit notification:new, incrémentez immédiatement la valeur locale de +1 sans faire de requête HTTP. Lorsque l'utilisateur consulte la liste, appelez PATCH /api/v1/notifications/read-all pour remettre le compteur à 0.

4. Module 3 : Notifications Push Firebase (FCM)

Les notifications Push permettent de réveiller le smartphone d'un client même lorsque l'application est fermée. Le backend envoie les messages au format Multicast HTTP v1 de Firebase en ciblant les tokens enregistrés dans la table UserDevice.

Cycle de Vie & Flux d'Enregistrement du Token

• **1. Demande de Permission :** Dès l'installation ou la première connexion, demander l'autorisation système (obligatoire sur iOS et Android 13+).

• **2. Récupération du FCM Token :** Obtenir la clé unique délivrée par Google via FirebaseMessaging.instance.getToken().

• **3. Enregistrement auprès du Backend :** Appeler POST /api/v1/notifications/devices avec le token et l'identifiant unique de l'appareil.

• **4. Rafraîchissement automatique :** Écouter onTokenRefresh pour mettre à jour le token si Firebase le renouvelle.

• **5. Dissociation lors du Logout :** Appeler DELETE /api/v1/notifications/devices/:deviceId pour empêcher la réception de messages après déconnexion.

Nouveaux Endpoints Backend Dédiés aux Appareils (FCM)

Méthode & Route	Corps de Requête (JSON)	Description & Sécurité
POST /api/v1/notifications/devices	{ "deviceId": "uuid-materiel", "platform": "android|ios|web", "pushToken": "fK9s...vKpQ", "appVersion": "1.0.0" }	Enregistre ou met à jour le token FCM de l'appareil connecté. Requiert l'en-tête Authorization: Bearer <TOKEN>. Gère automatiquement les doublons grâce à un upsert sur (userId, deviceId).
DELETE /api/v1/notifications/devices/:deviceId	Aucun (Paramètre dans l'URL)	Supprime l'appareil de la base de données. Doit obligatoirement être appelé lors du bouton "Se Déconnecter" (Logout).

Gestion des 3 États de l'Application Mobile

• **1. État Foreground (App au premier plan) :** Par défaut, Firebase ne fait pas vibrer ni n'affiche de bannière. Le listener onMessage reçoit l'alerte : le frontend doit afficher un SnackBar, un bandeau In-App ou un dialogue personnalisé.

• **2. État Background (App en arrière-plan) :** Le système d'exploitation gère automatiquement l'affichage natif de la bannière. Lors du clic utilisateur, l'événement onMessageOpenedApp est déclenché.

• **3. État Terminated (App complètement tuée) :** Le clic sur la notification relance l'application. L'événement getInitialMessage() fournit les métadonnées (ticketId, bookingId) pour rediriger l'utilisateur vers le bon écran.

Service Flutter Complet pour Push Notifications

```
import 'dart:io';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:device_info_plus/device_info_plus.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class PushNotificationService {
  final FirebaseMessaging _fcm = FirebaseMessaging.instance;
  final DeviceInfoPlugin _deviceInfo = DeviceInfoPlugin();

  Future<void> initializeAndRegister(String accessToken) async {
    // 1. Demander permission
    NotificationSettings settings = await _fcm.requestPermission(
      alert: true, badge: true, sound: true,
    );
    if (settings.authorizationStatus != AuthorizationStatus.authorized) return;

    // 2. Token FCM & ID Unique Appareil
    String? token = await _fcm.getToken();
    String deviceId = await _getDeviceId();
    String platform = Platform.isIOS ? 'ios' : 'android';

    if (token != null) {
      await _syncDeviceWithBackend(accessToken, deviceId, platform, token);
    }

    // 3. Écoute du renouvellement de token
    _fcm.onTokenRefresh.listen((newToken) {
      _syncDeviceWithBackend(accessToken, deviceId, platform, newToken);
    });

    // 4. Écoute en Foreground (Application ouverte)
    FirebaseMessaging.onMessage.listen((RemoteMessage msg) {
      print('Push reçu au premier plan : ${msg.notification?.title}');
    });

    // 5. Clic utilisateur pour redirection
    FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage msg) {
      _routeFromNotification(msg.data);
    });
  }

  Future<void> _syncDeviceWithBackend(String jwt, String devId, String plat, String tok) async {
    await http.post(
      Uri.parse('https://api.votredomaine.com/api/v1/notifications/devices'),
      headers: {'Content-Type': 'application/json', 'Authorization': 'Bearer ' + jwt},
      body: jsonEncode({
        'deviceId': devId, 'platform': plat, 'pushToken': tok, 'appVersion': '1.0.0',
      }),
    );
  }

  Future<void> unregisterDevice(String jwt) async {
    String deviceId = await _getDeviceId();
    await http.delete(
      Uri.parse('https://api.votredomaine.com/api/v1/notifications/devices/' + deviceId),
      headers: {'Authorization': 'Bearer ' + jwt},
    );
  }

  Future<String> _getDeviceId() async {
    if (Platform.isAndroid) {
      return (await _deviceInfo.androidInfo).id;
    }
    return (await _deviceInfo.iosInfo).identifierForVendor ?? 'ios-device';
  }

  void _routeFromNotification(Map<String, dynamic> data) {
    if (data['ticketId'] != null) {
      // Naviguer vers l'écran du ticket
    }
  }
}
```

5. Module 4 : SMS & Authentification Firebase OTP

Le projet applique une séparation stratégique stricte entre les SMS d'authentification/sécurité (100% gratuits) et les SMS transactionnels métiers.

Comparatif Stratégique des Deux Systèmes SMS

Fonctionnalité	Vérification & Reset MDP	SMS Transactionnels Salons
Fournisseur	Firebase Phone Auth (Google)	Termii SMS Gateway
Coût pour le Projet	100% Gratuit (10 000 SMS / mois)	Facturé au volume (opérateurs locaux)
Acteur à l'Initiative	Le SDK Frontend Mobile / Web	Le Backend Automatique (BullMQ)
Vérification	Preuve cryptographique via firebaseIdToken	Numéro normalisé +225 (CI)
Cas d'Application	• Réinitialisation de mot de passe • Validation premier numéro	• Rappel RDV 2h avant • Alerte No-Show / Annulation

Flux Firebase Phone Auth OTP (Réinitialisation Mot de Passe)

1. Le frontend saisit le numéro au format international obligatoire (+2250700000000).
2. Le frontend appelle verifyPhoneNumber du SDK Firebase.
3. Google expédie gratuitement le SMS contenant le code à 6 chiffres.
4. L'utilisateur tape le code reçu dans l'interface.
5. Firebase authentifie le code et génère un jeton signé idToken.
6. Le frontend envoie le idToken au backend via POST /api/v1/auth/reset-password.
7. Le backend valide le jeton auprès de Google, met à jour le mot de passe et révoque toutes les sessions compromises.

Code Flutter de Réinitialisation de Mot de Passe

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

final FirebaseAuth _auth = FirebaseAuth.instance;
String? _verificationId;

// 1. Déclencher le SMS gratuit
Future<void> sendOtp(String rawPhone) async {
  await _auth.verifyPhoneNumber(
    phoneNumber: rawPhone, // Format obligatoire: "+2250701020304"
    timeout: const Duration(seconds: 60),
    verificationCompleted: (cred) => resetPasswordWithCredential(cred, 'NouveauMdp!'),
    verificationFailed: (e) => print('Erreur SMS: ${e.message}'),
    codeSent: (verifId, _) => _verificationId = verifId,
    codeAutoRetrievalTimeout: (verifId) => _verificationId = verifId,
  );
}

// 2. Valider le code à 6 chiffres tapé et appeler le backend
Future<void> verifyAndReset(String code, String newPassword) async {
  final cred = PhoneAuthProvider.credential(
    verificationId: _verificationId!,
    smsCode: code,
  );
  await resetPasswordWithCredential(cred, newPassword);
}

Future<void> resetPasswordWithCredential(PhoneAuthCredential cred, String newMdp) async {
  final userCred = await _auth.signInWithCredential(cred);
  final idToken = await userCred.user?.getIdToken(true);

  final res = await http.post(
    Uri.parse('https://api.votredomaine.com/api/v1/auth/reset-password'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'firebaseIdToken': idToken, 'newPassword': newMdp}),
  );

  if (res.statusCode == 200) {
    print('Mot de passe réinitialisé avec succès !');
  }
}
```

6. Checklist Finale d'Intégration Frontend

Pour garantir une mise en production sans faille et sans étape oubliée, vérifiez chaque point ci-dessous avant toute validation QA :

1. Module Temps Réel (WebSockets / Socket.IO)

- [] **Connexion Namespace :** Vérifier que la connexion pointe bien vers /queue et non /.
- [] **Reconnexion automatique :** Configurer reconnection: true avec délai exponentiel (1s -> 5s).
- [] **Gestion Authentification :** Envoyer le Bearer JWT dans auth.token pour les mobiles, ou ?salonId pour la TV.
- [] **Sonnerie Carillon Audio :** Tester la lecture du son chime.mp3 lors de l'événement ticket:called avec playChime: true.
- [] **Invalidation Cache :** Brancher l'événement queue:updated sur le rafraîchissement des listes de tickets.

2. Notifications In-App & Centre de Notifications

- [] **Compteur Badge Rouge :** Appeler GET /api/v1/notifications/unread-count au démarrage de l'application.
- [] **Mise à jour en direct :** Incrémenter le badge lors de la réception de l'événement WebSocket notification:new.
- [] **Lecture à l'ouverture :** Appeler PATCH /api/v1/notifications/:id/read au clic sur un élément.
- [] **Écran Préférences :** Permettre à l'utilisateur d'activer/désactiver les canaux dans ses paramètres.

3. Notifications Push Firebase (FCM)

- [] **Déclarations OS :** Fichiers google-services.json (Android) et GoogleService-Info.plist (iOS) installés.
- [] **Permissions Requises :** Demander la permission système dès la fin du tutoriel / onboarding.
- [] **Enregistrement Backend :** Appeler POST /api/v1/notifications/devices dès que l'utilisateur est connecté.
- [] **Écoute Token Refresh :** Ré-enregistrer le token automatiquement sur l'événement onTokenRefresh.
- [] **Nettoyage au Logout :** Appeler obligatoirement DELETE /api/v1/notifications/devices/:deviceId à la déconnexion.
- [] **Redirection Métier :** Vérifier que le clic sur la notification redirige vers le bon écran (ticketId / bookingId).

4. SMS & Authentification Firebase OTP

- [] **Console Firebase :** Activer le fournisseur Téléphone dans la console Firebase Authentication.
- [] **Empreintes SHA :** Renseigner les clés SHA-1 et SHA-256 dans Firebase pour éviter le captcha Web sur Android.
- [] **Numéro International :** S'assurer que le frontend formate toujours le numéro avec le préfixe +225.
- [] **Sécurité idToken :** Ne jamais envoyer le code OTP en clair au backend, uniquement le firebaseIdToken signé.

> **CONSEIL DE VALIDATION EN ÉQUIPE**
> Avant la livraison aux bêta-testeurs, réalisez un test de bout en bout : Prenez un ticket d'attente, fermez l'application, appelez le ticket depuis l'espace coiffeur et observez l'arrivée de la notification Push, le son sur la TV du salon et la mise à jour du badge à la réouverture.